UADE

UADE

Programación III – Diseño y Análisis de Algoritmos

Temas a desarrollar

1. Repaso clase anterior
2. PRIM
3. Kruskal
4. Resolución de Ejercicios

Repaso Clase Anterior

UADE

Greedy Grafos - Dijsktra

Problema de los caminos mínimos:

> *Dado un grafo y un nodo particular del mismo, buscar el costo del camino mas corto entre ese nodo y todos los demás, siempre que exista un camino.*

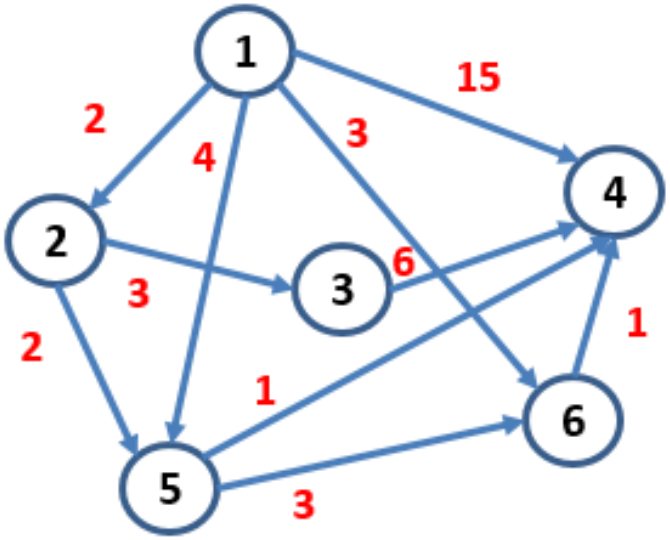
El resultado es un grafo con los mismos nodos que el original pero solo hay aristas entre el nodo buscado y todos los demás y el peso de la arista es el costo del camino mínimo.

Para poder ejecutar el algoritmo es necesario contar con un grafo dirigido, con todas sus aristas con peso positivo.

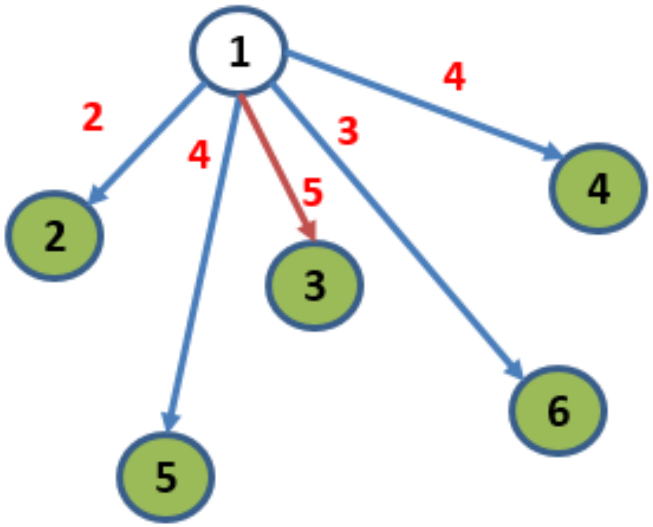
La complejidad temporal del algoritmo es $O(n^2)$

Greedy Grafos – Dijsktra - Ejemplo

Grafo Original



Grafo Resultado FINAL



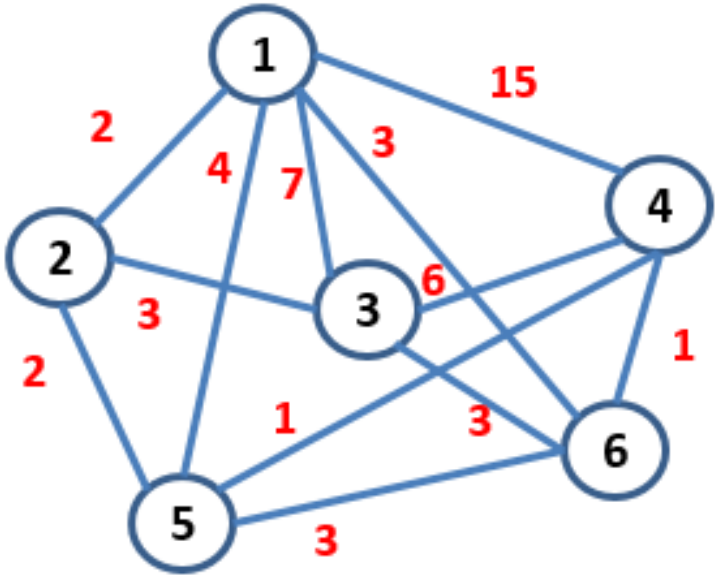
UADE

Árbol de recubrimiento de costo mínimo

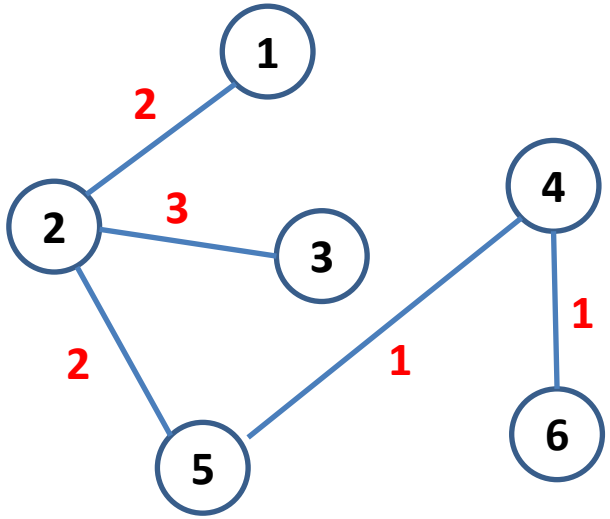
- Los árboles de recubrimiento se trabajan con grafos no dirigidos.
- Un árbol de recubrimiento de un grafo *G(V;A)* a un grafo *G'(V;A')* que contiene los mismos vértices que *G* y un subconjunto de las aristas, que forma un árbol, es decir, *G'* es un grafo conexo y acíclico.
- Árbol de recubrimiento mínimo es al árbol de recubrimiento que tenga el menor costo de todos, es decir que la suma de sus aristas sea la mínima.
- Para algunos grafos, puede existir más de un árbol de recubrimiento mínimo, todos con el mismo costo.

Árbol de Recubrimiento Mínimo – Ejemplo

Grafo original



Ej. árbol recubrimiento



Costo: 9

Es el árbol de recubrimiento de costo mínimo

UADE

Árbol de Recubrimiento Mínimo - Algoritmos

- Existen dos algoritmos con estrategia ***Greedy*** que permite obtener un árbol de cubrimiento de un grafo con costo mínimo.

- Los algoritmos son:
  - **PRIM**
  - **Kruskal**

- En el caso de que se tenga un único árbol de recubrimiento de costo mínimo, ambos algoritmos arrojan el mismo resultado. Si no, podría variar por la estrategia que sigue cada uno.

Qué es el algoritmo de PRIM

Algoritmo de PRIM

La estrategia del algoritmo de PRIM para hallar el árbol de recubrimiento mínimo consiste en ir armando un árbol a partir de un vértice cualquiera del grafo, e ir agregando vértices hasta completar todos sus nodos.

Estos vértices son seleccionados por el criterio que tenga un menor costo con otro nodo previamente seleccionado.

Algoritmo de PRIM - Estrategia

- **Candidatos**: los nodos del grafo.

- **Función de selección**: la estrategia para ir agregando vértices es ir agregando el nodo que, no estando agregado aún, sea el que tenga la arista de costo mínimo con algún otro vértice ya seleccionado.

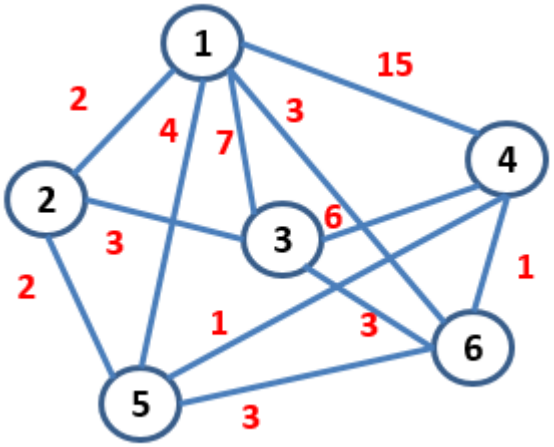
- **La función factibilidad:** dependiendo la implementación es decidir si se utiliza la arista obtenida en la selección o no.

- **La función solución**: haber trabajado con todos los nodos candidatos.

- **La función objetivo:** árbol de recubrimiento de costo mínimo.

Algoritmo de PRIM - Ejemplo

Grafo original



Árbol recubrimiento mínimo



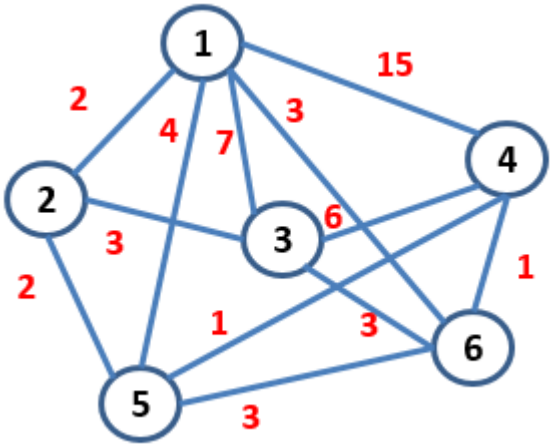
Tratados	*Pendientes*
{}	{1,2,3,4,5,6}

Selecciono = 2

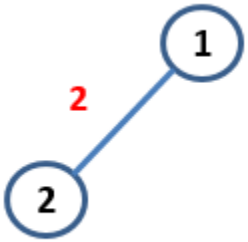
UADE

Algoritmo de PRIM - Ejemplo

Grafo original



Árbol recubrimiento mínimo



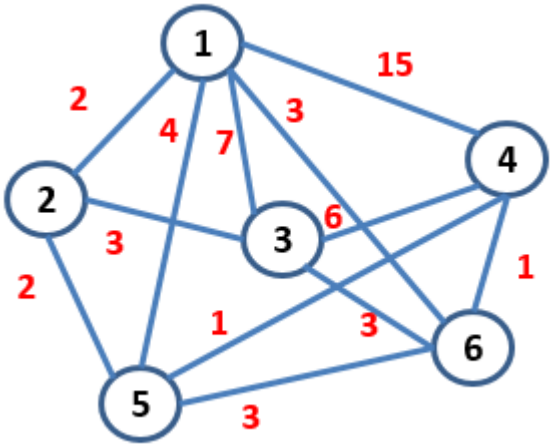
Tratados	***Pendientes***
{2}	{1,3,4,5,6}

Selecciono = 1 porque es el de menor costo entre tratados y pendientes

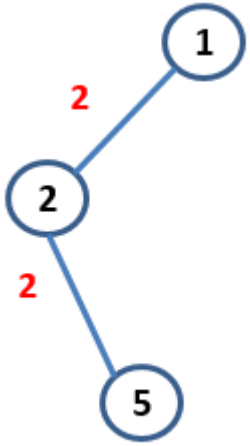
También se puede seleccionar el 5 porque tiene el mismo costo de arista

Algoritmo de PRIM - Ejemplo

Grafo original



Árbol recubrimiento mínimo



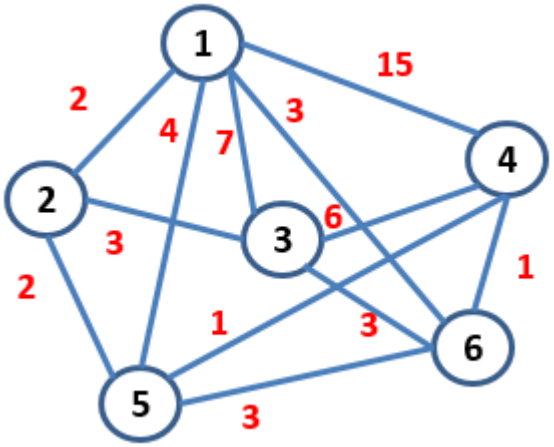
Tratados	*Pendientes*
{2,1}	{3,4,5,6}

Selecciono = 5 porque es el de menor costo entre tratados y pendientes

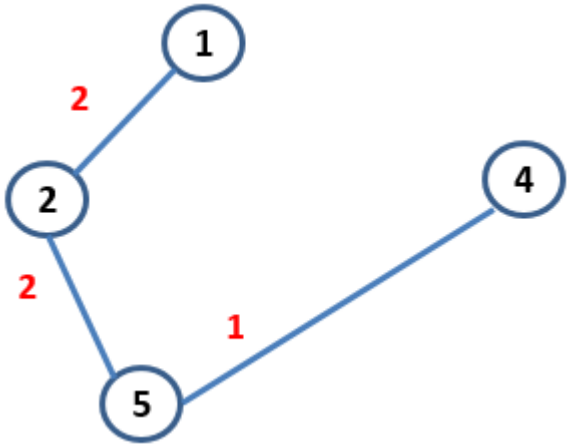
UADE

Algoritmo de PRIM - Ejemplo

Grafo original



Árbol recubrimiento mínimo



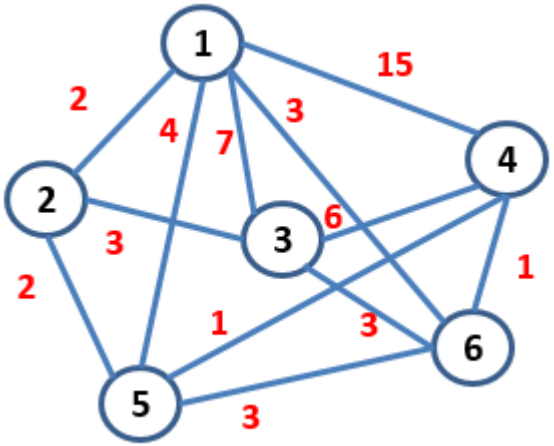
Tratados	***Pendientes***
{2,1,5}	{3,4,6}

Selecciono = 4 porque es el de menor costo entre tratados y pendientes

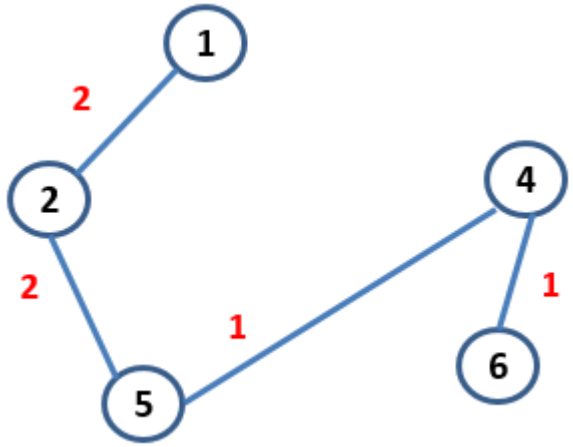
UADE

Algoritmo de PRIM - Ejemplo

Grafo original



Árbol recubrimiento mínimo

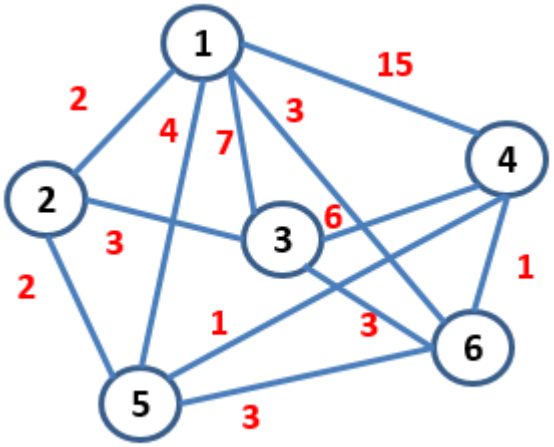


Tratados	***Pendientes***
{2,1,5,4}	{3,6}

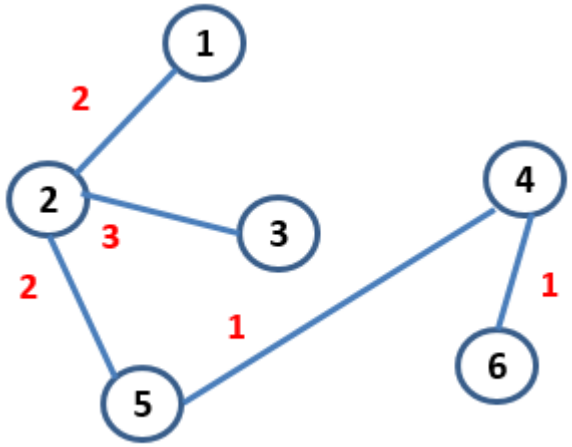
Selecciono = 6 porque es el de menor costo entre tratados y pendientes

Algoritmo de PRIM - Ejemplo

Grafo original



Árbol recubrimiento mínimo



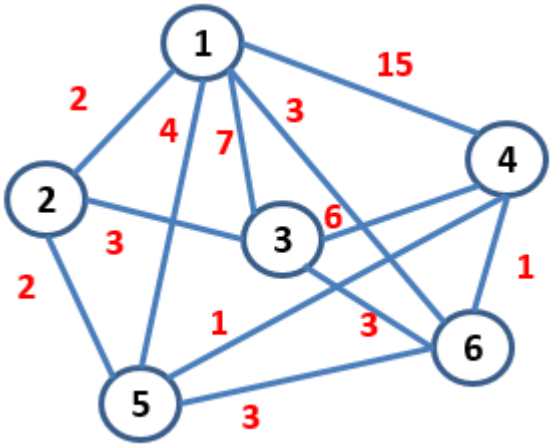
Tratados	*Pendientes*
{2,1,5,4,6}	{3}

Selecciono = 3 porque es el de menor costo entre tratados y pendientes
en éste caso hay dos opciones porque hay dos aristas con costo 3.
Lo anterior implica que hay más de un árbol de recubrimiento de costo mínimo

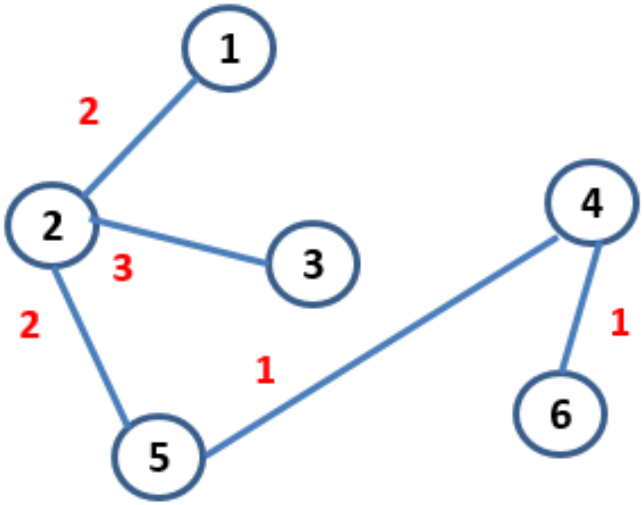
UADE

Algoritmo de PRIM - Ejemplo

Grafo original



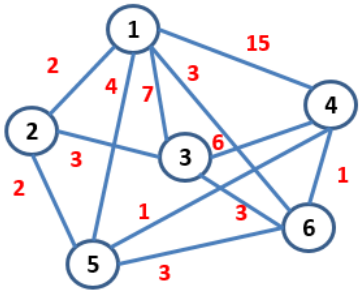
Árbol recubrimiento mínimo



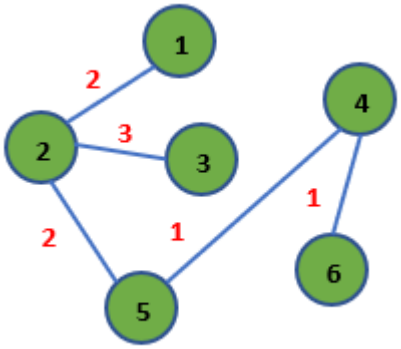
Costo Árbol Recubrimiento Mínimo= 9

UADE

Algoritmo de PRIM - Algoritmo




```
ALGORITMO PRIM
Entrada: G: grafo de origen
 alida: R: grafo solución
 inicializar(R)
 para cada v ∈ Vertices(G)
   agregarVertice(R, v)
 fin para
 vertice ← elegir(Vertices(R))
 pendientes ← Vertices(R) \ {vertice}
 para cada w ∈ Adyacentes(G, vertice)
   agregarArista(R, vertice, w, pesoArista(G, vertice, w))
 fin para
 mientras pendientes ≠ ∅
   vertice = v : pesoArista(v, v_i) = mín(pesoArista(v_1, v_2) : ∀v_1 ∈
   pendientes, v_2 ∉ pendientes
   sacar(pendientes, vertice)
   para cada p ∈ pendientes
     si existeArista(G, p, vertice) Y NO existeArista(R, p, x)∀x
       agregarArista(R, p, vertice, pesoArista(G, p, vertice))
     sino
       si existeArista(G, p, vertice) Y pesoArista(R, p, x) >
       pesoArista(G, p, vertice)
         eliminarArista(R, p, x)
         agregarArista(R, p, vertice, pesoArista(G, p, vertice))
       fin si
     fin si
   fin para
 fin mientras
 devolver R
```



UADE

Algoritmo de PRIM – Complejidad Temporal

ALGORITMO PRIM
Entrada: G: grafo de origen
Salida: R: grafo solución

```
inicializar(R)
para cada v ∈ Vertices(G)
    agregarVertice(R, v)
fin para
vertice ← elegir(Vertices(R))
pendientes ← Vertices(R) \ {vertice}
para cada w ∈ Adyacentes(G, vertice)
    agregarArista(R, vertice, w, pesoArista(G, vertice, w))
fin para
mientras pendientes ≠ ∅
    vertice = v : pesoArista(v, v_i) = mín(pesoArista(v_1, v_2) : ∀v_1 ∈
    pendientes, v_2 ∉ pendientes
    sacar(pendientes, vertice)
    para cada p ∈ pendientes
        si existeArista(G, p, vertice) Y NO existeArista(R, p, x)∀x
            agregarArista(R, p, vertice, pesoArista(G, p, vertice))
        sino
            si existeArista(G, p, vertice) Y pesoArista(R, p, x) >
            pesoArista(G, p, vertice)
                eliminarArista(R, p, x)
                agregarArista(R, p, vertice, pesoArista(G, p, vertice))
            fin si
        fin si
    fin para
fin mientras
devolver R
```

Los ciclos dependen de la cantidad de vértices, teniendo un ciclo anidado.

$$O(n^2)$$

Qué es
el algoritmo de Kruskal

UADE

Algoritmo de Kruskal

- El algoritmo de Kruskal se basa en ir armando árboles pequeños e ir uniéndolos hasta obtener un único árbol de recubrimiento cuyo costo sea mínimo.

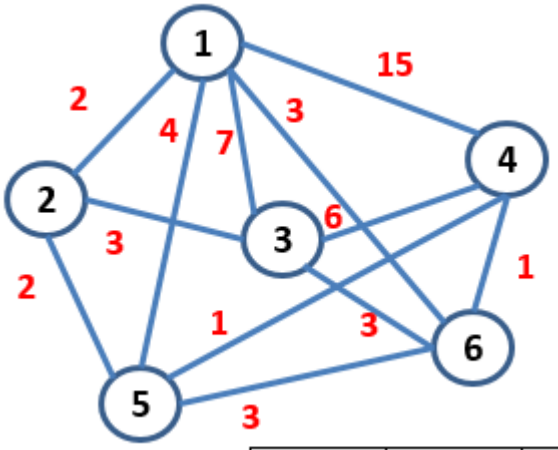
- Se comienza armando un árbol con un único nodo por cada vértice. Luego se toman las aristas del grafo original, ordenadas por su peso, de menor a mayor. Para cada arista, si tiene cada extremo en un árbol diferente, se agrega dicha arista al resultado y se unen los dos árboles que contienen sus extremos.

Algoritmo de Kruskal - Estrategia

- **Candidatos:** todas las aristas del grafo.
- **Función de selección:** seleccionar la arista de menor valor.
- **La función factibilidad:** la arista no debe tener los nodos en un mismo subárbol.
- **La función solución**: haber trabajado con todas las aristas.
- **La función objetivo:** árbol de recubrimiento de costo mínimo.

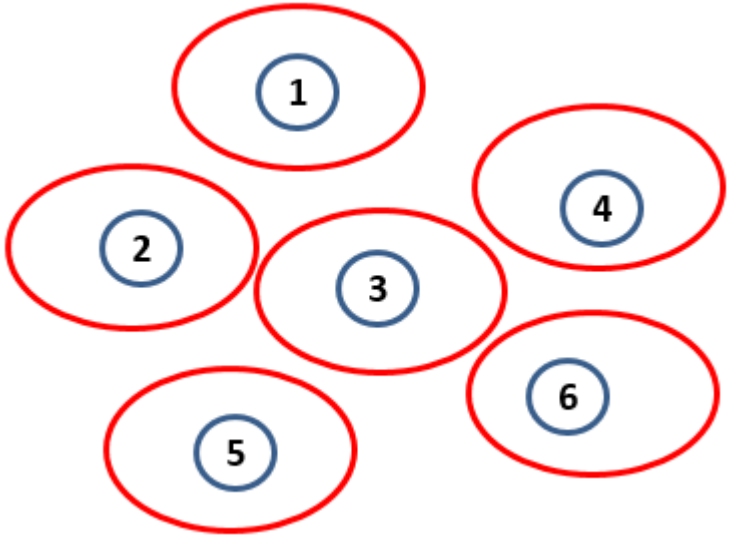
Algoritmo de Kruskal - Ejemplo

Grafo original



Origen	Destino	Costo
5	4	1
6	4	1
2	1	2
2	5	2
1	6	3
2	3	3
3	6	3
5	6	3
1	5	4
3	4	6
1	3	7
1	4	15

Árbol recubrimiento mínimo

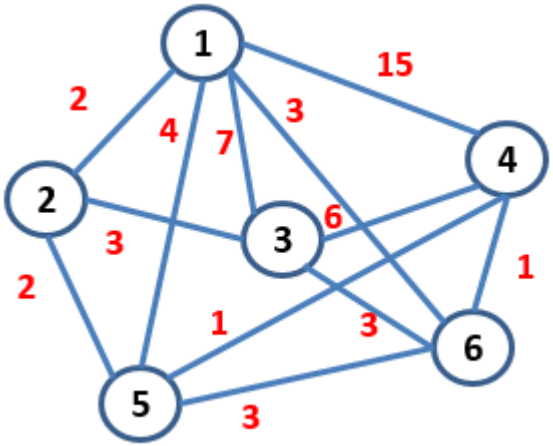


Inicialmente cada nodo está en un subárbol y las aristas ordenadas por costo de menor a mayor.

UADE

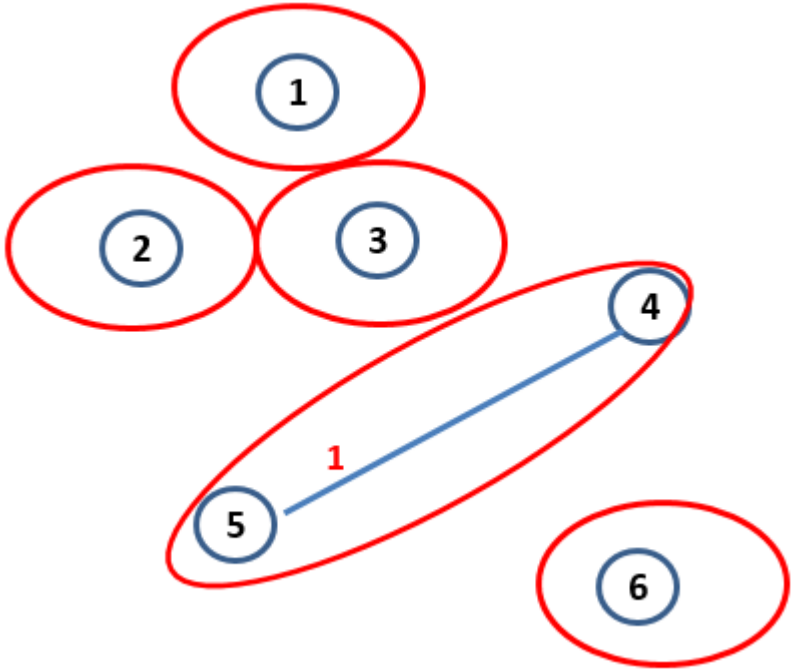
Algoritmo de Kruskal - Ejemplo

Grafo original



Origen	Destino	Costo
5	4	1
6	4	1
2	1	2
2	5	2
1	6	3
2	3	3
3	6	3
5	6	3
1	5	4
3	4	6
1	3	7
1	4	15

Árbol recubrimiento mínimo

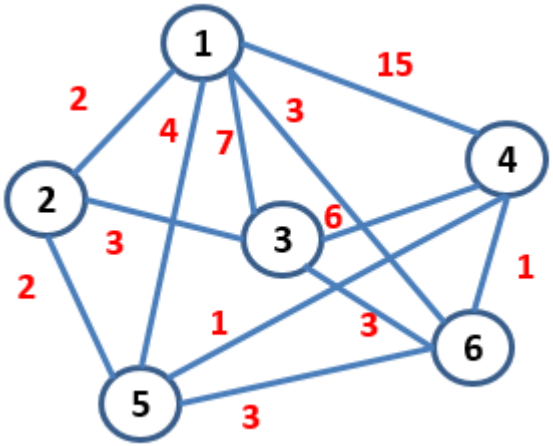


Se selecciona la arista de menor tamaño entre las que quedan y, si es factible, se utiliza.

UADE

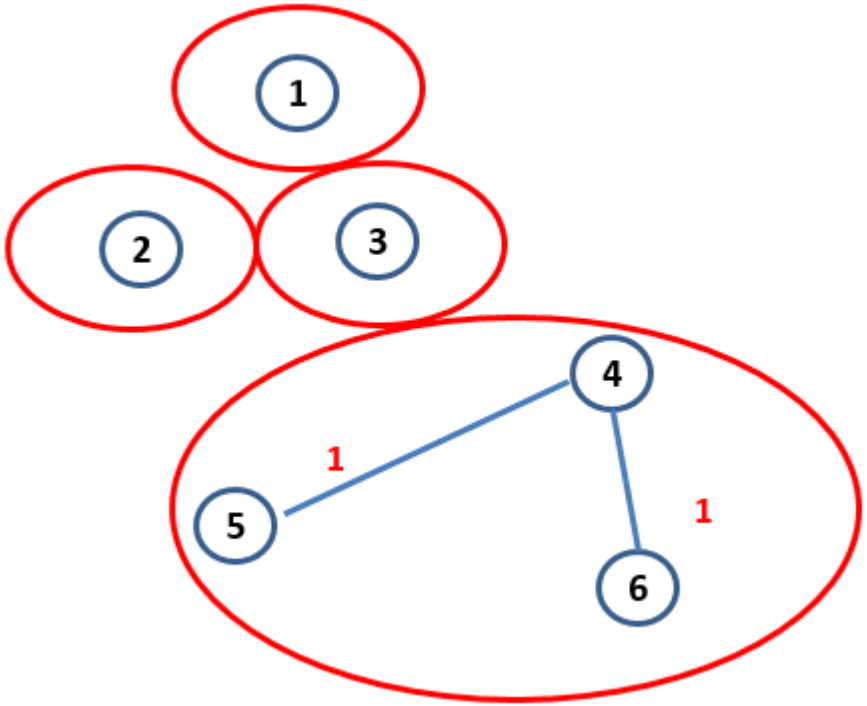
Algoritmo de Kruskal - Ejemplo

Grafo original



Origen	Destino	Costo
5	4	1
6	4	1
2	1	2
2	5	2
1	6	3
2	3	3
3	6	3
5	6	3
1	5	4
3	4	6
1	3	7
1	4	15

Árbol recubrimiento mínimo

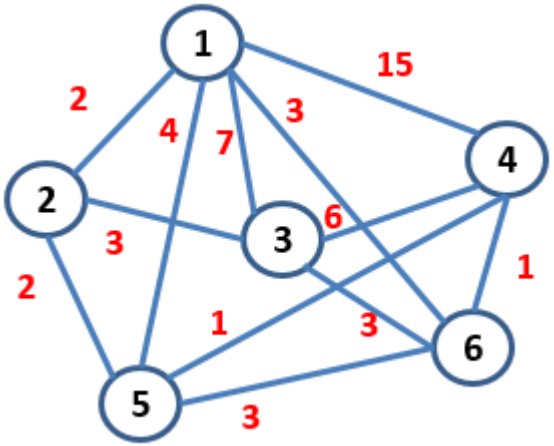


Se selecciona la arista de menor tamaño entre las que quedan y, si es factible, se utiliza.

UADE

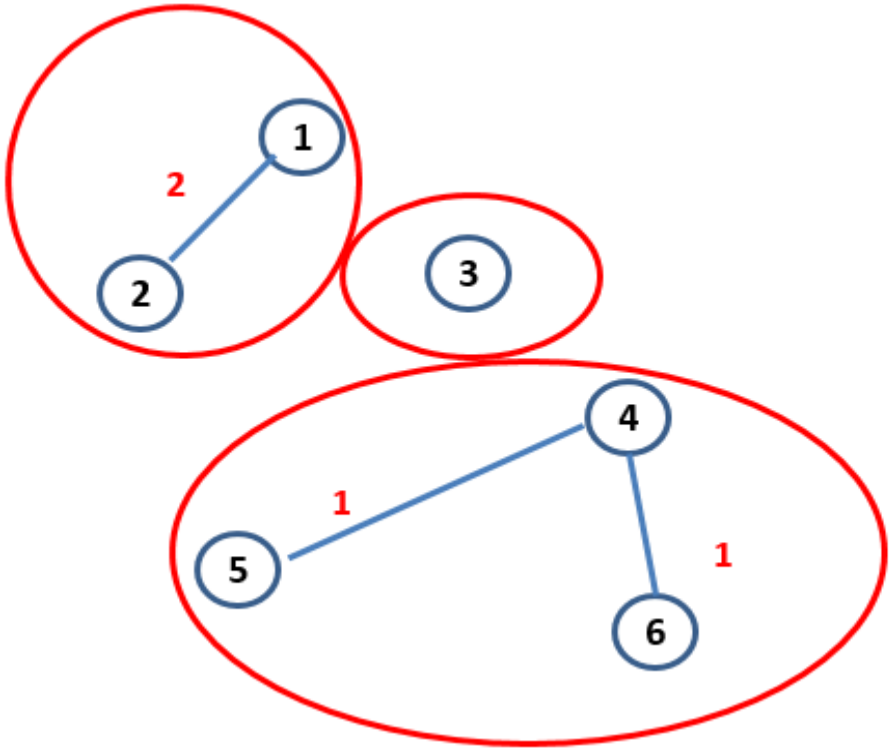
Algoritmo de Kruskal - Ejemplo

Grafo original



Origen	Destino	Costo
5	4	1
6	4	1
2	1	2
2	5	2
1	6	3
2	3	3
3	6	3
5	6	3
1	5	4
3	4	6
1	3	7
1	4	15

Árbol recubrimiento mínimo

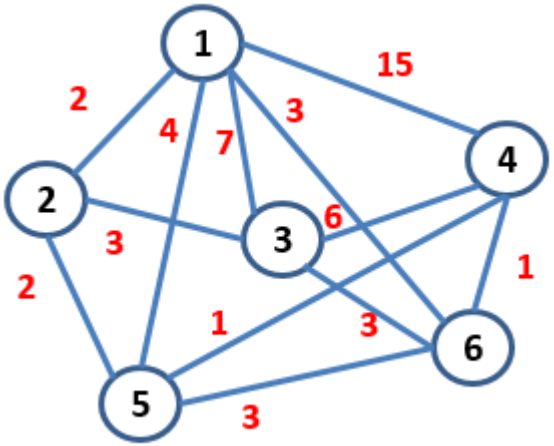


Se selecciona la arista de menor tamaño entre las que quedan y, si es factible, se utiliza.

UADE

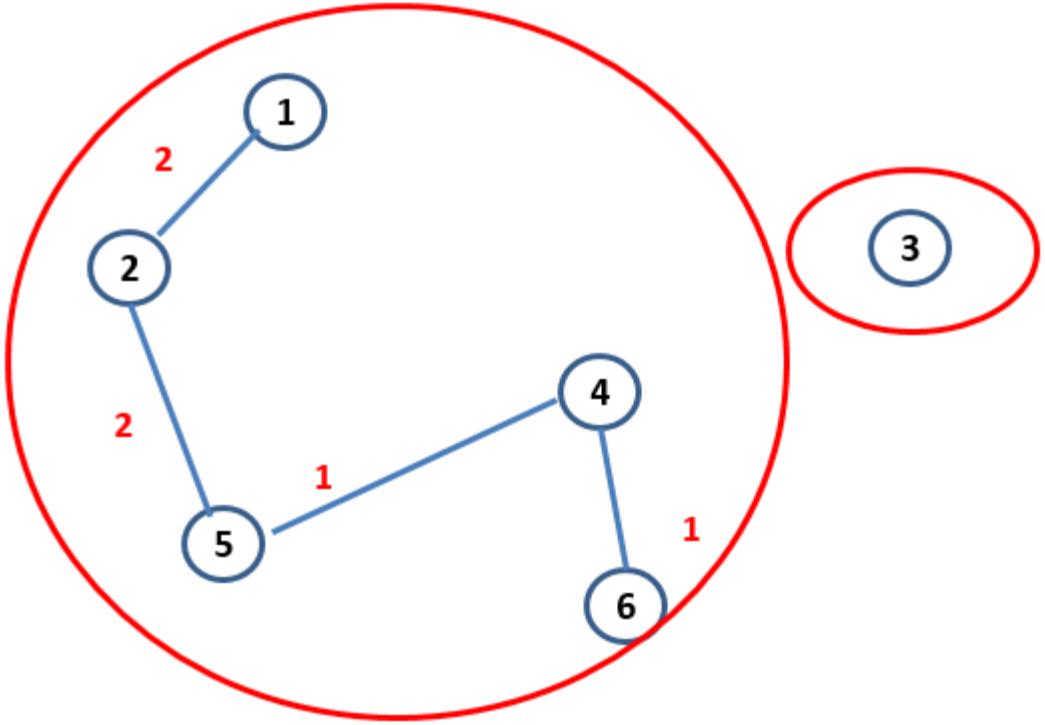
Algoritmo de Kruskal - Ejemplo

Grafo original



Origen	Destino	Costo
5	4	1
6	4	1
2	1	2
2	5	2
1	6	3
2	3	3
3	6	3
5	6	3
1	5	4
3	4	6
1	3	7
1	4	15

Árbol recubrimiento mínimo

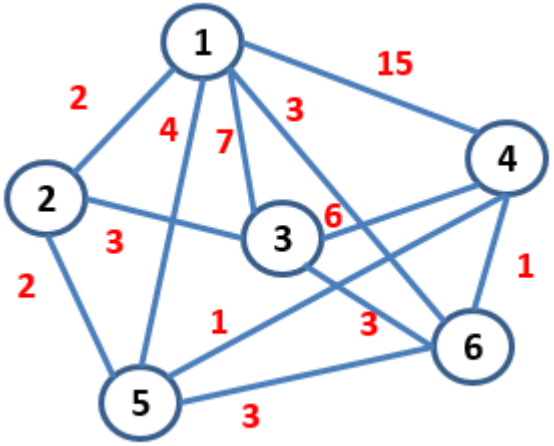


Se selecciona la arista de menor tamaño entre las que quedan y, si es factible, se utiliza.

UADE

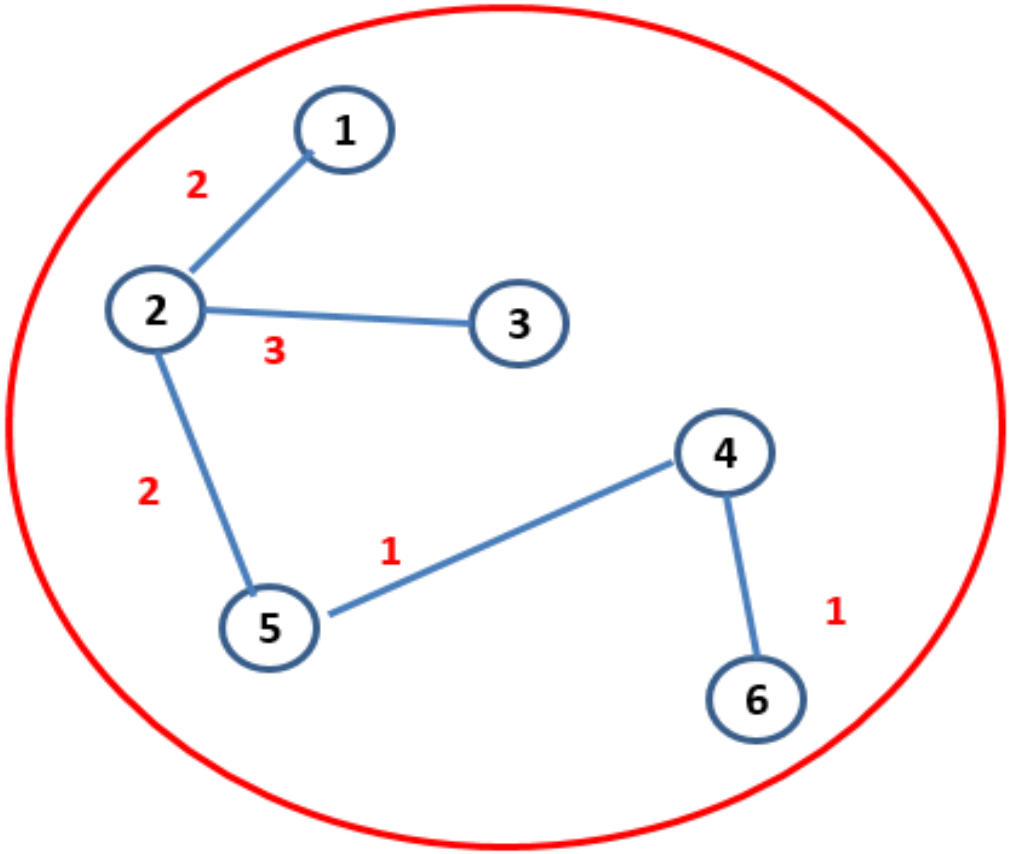
Algoritmo de Kruskal - Ejemplo

Grafo original



Origen	Destino	Costo
5	4	1
6	4	1
2	1	2
2	5	2
1	6	3
2	3	3
3	6	3
5	6	3
1	5	4
3	4	6
1	3	7
1	4	15

Árbol recubrimiento mínimo



Se selecciona la arista de menor tamaño entre las que quedan y, si es factible, se utiliza.

UADE

Algoritmo de Kruskal - Algoritmo

```
ALGORITMO KRUSKAL
Entrada: G: grafo de origen
Salida: R: grafo solución
  inicializar(R)
  E ← inicializarConjuntos()
  cantidadVertices ← 0
  para cada v ∈ Vertices(G)
    agregarVertice(R, v)
    E[v] = v
    cantidadVertices ← cantidadVertices + 1
  fin para
  C ← inicializarColaPrioridad()
  para cada v1 ∈ Vertices(G)
    para cada v2 ∈ Vertices(G)
      si existeArista(G, v1, v2) Y NO existe(C, Arista(v1, v2))
        agregar(C, Arista(v1, v2), pesoArista(G, v1, v2))
      fin si
    fin para
  fin para
```

Indice (vertice)	*Nro. Componente*
1	1
2	2
3	3
4	4
5	5
6	6

Continúa en próxima *slide*

Algoritmo de Kruskal – Algoritmo (Cont.)

Indice (vertice)	*Nro. Componente*
1	2
2	2
3	2
4	2
5	2
6	2

```
mientras cantidadVertices > 1
    a ← menor(C)
    eliminarMenor(C)
    si E[a.origen] <> E[a.destino]
        cantidadVertices ← cantidadVertices – 1 //unifico los conjuntos
        para cada v ∈ E
            si E[v] = E[a.destino]
                E[v] ← E[a.origen]
            fin si
        fin para
        agregarArista(R, a.origen, a.destino, a.peso)
    fin si
fin mientras
devolver R
```

Origen	Destino	Costo
5	4	1
6	4	1
2	1	2
2	5	2
1	6	3
2	3	3
3	6	3
5	6	3
1	5	4
3	4	6
1	3	7
1	4	15

Algoritmo de Kruskal – Complejidad Temporal

```
ALGORITMO KRUSKAL
Entrada: G: grafo de origen
Salida: R: grafo solución
  inicializar(R)
  E ← inicializarConjuntos()
  cantidadVertices ← 0
  para cada v ∈ Vertices(G)
    agregarVertice(R, v)
    E[v] = v
    cantidadVertices ← cantidadVertices + 1
  fin para
  C ← inicializarColaPrioridad()
  para cada v1 ∈ Vertices(G)
    para cada v2 ∈ Vertices(G)
      si existeArista(G, v1, v2) Y NO existe(C, Arista(v1, v2))
        agregar(C, Arista(v1, v2), pesoArista(G, v1, v2))
      fin si
    fin para
  fin para
```

Aquí itera por la cantidad de vértices.

Aquí itera por la cantidad de vértices, más el costo de agregar una arista a la cola de prioridad.

Continúa en próxima *slide*

Algoritmo de Kruskal – Complejidad Temporal (Cont.)

```
mientras cantidadVertices > 1
  a ← menor(C)
  eliminarMenor(C)
  si E[a.origen] <> E[a.destino]
    cantidadVertices ← cantidadVertices − 1 //unifico los conjuntos
    para cada v ∈ E
      si E[v] = E[a.destino]
        E[v] = E[a.origen]
      fin si
    agregarArista(R, a.origen, a.destino, a.peso)
  fin si
fin mientras
devolver R
```

Aquí itera por la cantidad de vértices.

$\Theta(n^2 \log a)$ donde **"n"** es la cantidad de vértices y **"a"** la cantidad de aristas, sí la cantidad de aristas tiende a n^2 entonces se puede decir que la complejidad es:

$\Theta(a \log a)$

Resolución de Ejercicios

Ejercicios

Lea detenidamente cada problema, por cada uno de ellos deberá responder los siguientes puntos:

d) Determinar qué algoritmo sobre grafos aplicaría.

e) Pseudocódigo del Algoritmo de la resolución del problema, el cual debe seguir la estrategia definida en el punto a)

f) Cálculo de la Complejidad Temporal, justificando la misma en forma detallada.

2. Dado un GrafoTDA **G** dirigido, cuyos nodos representan la ciudad de Buenos Aires y las ciudades capitales de las provincias de la Argentina, y las aristas entre esos nodos los tiempos de vuelo entre las ciudades. Determinar cuáles ciudades convienen ser alcanzadas desde la ciudad de Buenos Aires a través de otra ciudad, y no en forma directa. Suponer que el tiempo de demora de aterrizaje es despreciable.

Ejercicios

3. Las emisiones de los automóviles privados son una de las principales fuentes de contaminación en las grandes ciudades. Las autoridades universitarias han decidido contribuir seriamente a la mejora de la calidad del aire prohibiendo el paso de automóviles por el campus y creando una serie de líneas de autobuses eléctricos que permitan acceder a distintos puntos de la Ciudad Universitaria. Para ello se han estudiado los principales flujos de vehículos entre distintos puntos seleccionados facultades, estación de metro, etc.) de la Ciudad Universitaria. Los resultados del estudio contienen el número de vehículos que transitan al día entre estos puntos seleccionados (sin tener en cuenta el sentido en el que circulan). El objetivo de este estudio es localizar los trayectos entre puntos seleccionados que utilizan más vehículos y que permiten conectar todos los puntos seleccionados. Diseñar un algoritmo que proporcione estos trayectos. Validar que el algoritmo sea óptimo en base a algoritmos conocidos.

¡Muchas gracias!

UADE